

EC200U&EG91xU Series

QuecOpen(SDK) FTP(S) API

Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-09-26

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-02-23	Larson LI	Creation of the document
1.0	2022-04-29	Larson LI/ Kruskal ZHU	First official release
1.1	2023-09-26	Kruskal ZHU	1. Updated the document name based on the unified QuecOpen naming. 2. Added applicable modules EG912U-GL and EG915U series. 3. Updated the enumeration of FTP(S) error codes (Chapter 3.3.1.1) 4. Updated the enumeration of attribute tags or attribute parameters (Chapter 3.3.4.1)

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules	6
2 FTP(S) API Calling Process	7
3 FTP(S) API.....	9
3.1. Header File.....	9
3.2. API Overview	9
3.3. API Description	10
3.3.1. Error Code Enumeration	10
3.3.1.1. QL_FTP_CLIENT_ERR_E.....	10
3.3.2. ql_ftp_client_new	12
3.3.3. ql_ftp_client_release	13
3.3.4. ql_ftp_client_setopt	13
3.3.4.1. QL_FTP_CLIENT_OPT_E.....	14
3.3.5. ql_ftp_client_open.....	15
3.3.6. ql_ftp_client_close.....	16
3.3.7. ql_ftp_client_get_ex	16
3.3.7.1. QL_FTP_CLIENT_WRITE_CB_EX	17
3.3.8. ql_ftp_client_put_ex	18
3.3.8.1. QL_FTP_CLIENT_READ_CB_EX.....	19
3.3.9. ql_ftp_client_delete	20
3.3.10. ql_ftp_client_cwd.....	20
3.3.11. ql_ftp_client_lcwd.....	21
3.3.12. ql_ftp_client_pwd	21
3.3.13. ql_ftp_client_lpwd.....	22
3.3.14. ql_ftp_client_mkdir	22
3.3.15. ql_ftp_client_rmdir.....	23
3.3.16. ql_ftp_client_list.....	23
3.3.16.1. QL_FTP_CLIENT_FLIE_INFO_T	24
3.3.17. ql_ftp_client_size.....	25
3.3.18. ql_ftp_client_rename.....	25
4 Example	27
5 Appendix References	28

Table Index

Table 1: Applicable Modules	6
Table 2: API Overview	9
Table 3: Related Documents.....	28
Table 4: Terms and Abbreviations	28

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces the FTP(S) API, related calling process and example on Quectel EC200U series and EG91xU family modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 FTP(S) API Calling Process

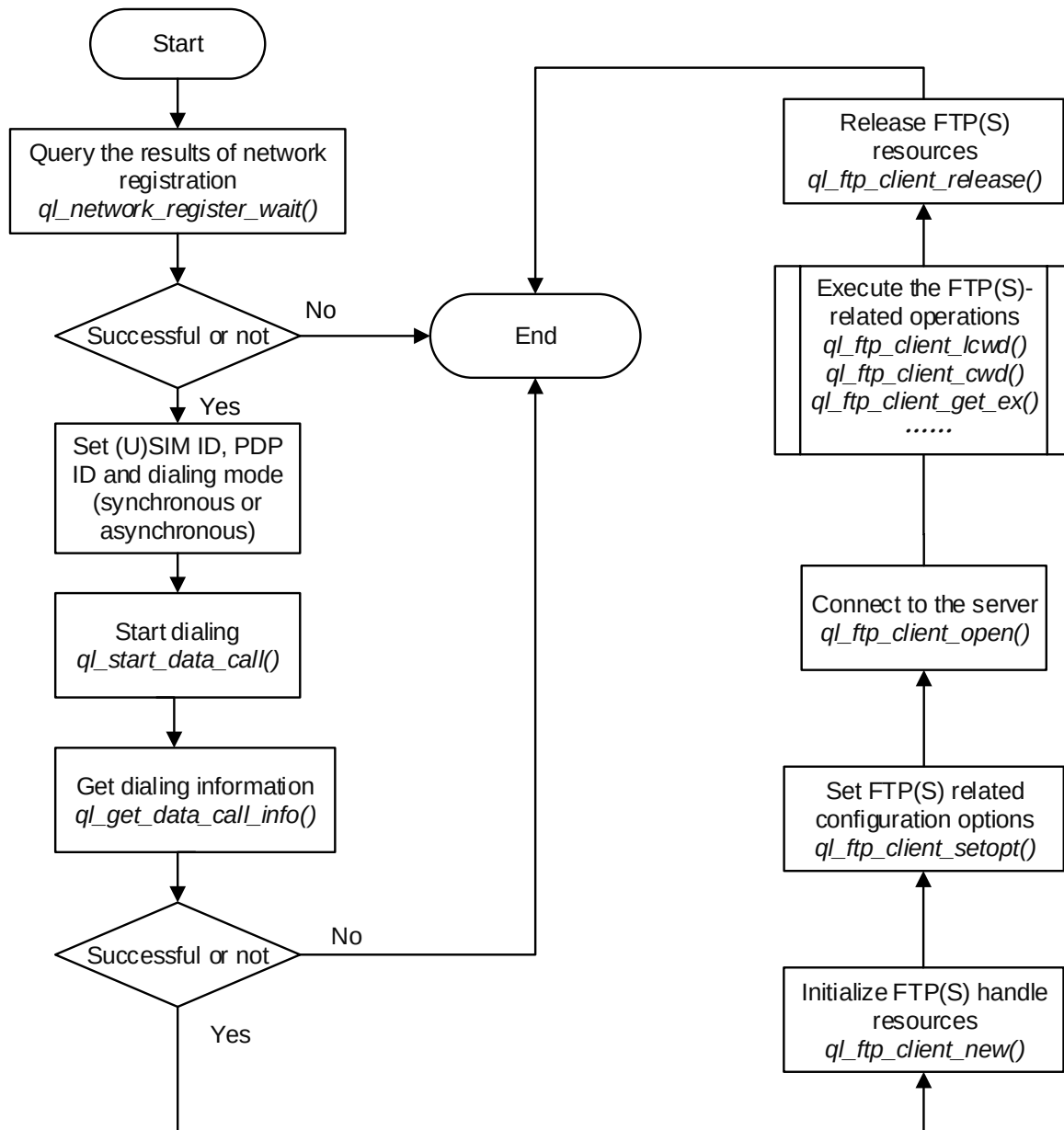


Figure 1: FTP(S) API Calling Process

Before sending an FTP(S) request, you need to establish a data channel based on which the FTP(S) sends and receives data and configure the network. The data channel establishment step can be omitted if it has been established in other tasks, but you need to get SIM_CID of the data channel, because the FTP(S) client uses the data channel through SIM_CID.

SIM_CID configuration method:

After creating an FTP(S) client handle through *ql_ftp_client_new()*, you can configure SIM_CID of the data channel used by the FTP(S) client and whether SSL encryption is required through *ql_ftp_client_setopt()*.

NOTE

1. The FTP(S) data channel supports two modes: active mode (PORT) and passive mode (PASV). The current default mode is PASV.
2. When creating an FTP(S) task through *ql_rtos_task_create()*, please allocate at least 16 KB of stack space so as to complete the basic features of FTP(S) function. See **document [2]** for detailed information about *ql_rtos_task_create()*.
3. See **document [3]** for detailed information about *ql_network_register_wait()*, *ql_start_data_call()* and *ql_get_data_call_info()*.

3 FTP(S) API

3.1. Header File

ql_ftp_client.h, the header file of FTP(S) API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_ftp_client_new()</i>	Creates an FTP(S) client handle and initializes FTP(S) client resources.
<i>ql_ftp_client_release()</i>	Releases FTP(S) client resources.
<i>ql_ftp_client_setopt()</i>	Configures FTP(S) client attributes.
<i>ql_ftp_client_open()</i>	Connects to the FTP(S) server.
<i>ql_ftp_client_close()</i>	Disconnects from the FTP(S) server.
<i>ql_ftp_client_get_ex()</i>	Downloads a file from the FTP(S) server.
<i>ql_ftp_client_put_ex()</i>	Uploads a local file or user data to the FTP(S) server.
<i>ql_ftp_client_delete()</i>	Deletes an FTP(S) server file.
<i>ql_ftp_client_cwd()</i>	Alters the current directory used by the FTP(S) server.
<i>ql_ftp_client_lcwd()</i>	Alters the local directory currently used.
<i>ql_ftp_client_pwd()</i>	Gets the current directory used by the FTP(S) server.
<i>ql_ftp_client_lpwd()</i>	Gets the local directory currently used.

<code>ql_ftp_client_mkdir()</code>	Creates a directory on the FTP(S) server.
<code>ql_ftp_client_rmdir()</code>	Deletes the directory on the FTP(S) server.
<code>ql_ftp_client_list()</code>	Gets the list of the specified file or directory on the FTP(S) server.
<code>ql_ftp_client_size()</code>	Gets the size of the specified file under the specified path on the FTP(S) server.
<code>ql_ftp_client_rename()</code>	Renames the specified file or directory on the FTP(S) server.

3.3. API Description

3.3.1. Error Code Enumeration

3.3.1.1. QL_FTP_CLIENT_ERR_E

The enumeration of FTP(S) error codes:

```
typedef enum
{
    QL_FTP_ERR_OK = 0,
    QL_FTP_ERR_UNKNOWN = 601 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_WOULDBLOCK = 602 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_BUSY = 603 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_DNS = 604 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_NETWORK = 605 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_CTRL_CLOSE = 606 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_DATA_CLOSE = 607 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_BEARER_FAIL = 608 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_TIMEOUT = 609 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_INVALID_PARAM = 610 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_FILE_NOT_FOUND = 611 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_FILE_POINT_ERR = 612 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_FS_FILE_ERROR = 613 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_SERVICE_END = 614 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_DATA_FAILED = 615 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_CLOSE_CONN = 616 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_FILE_UNOPER = 617 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_REQUEST_ABORT = 618 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_OVER_MEMORY = 619 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_COMMAND_ERROR = 620 | QL_FTP_ERRCODE_BASE,
    QL_FTP_ERR_PARAM_ERROR = 621 | QL_FTP_ERRCODE_BASE,
```

```

QL_FTP_ERR_COMMAND_FAILED          = 622 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_COMMAND_BAD_SEQUENCE    = 623 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_COMMAND_NOT_IMPLMENT    = 624 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_UNLOGIN                  = 625 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_NO_ACCOUNT               = 626 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_REQUEST_FAILED           = 627 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_REQUEST_STOP             = 628 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_FILEREQ_STOP             = 629 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_FILENAME_ERROR           = 630 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_SSL_AUTH_FAIL            = 631 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_INTERFACE_FAILED         = 632 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_SEND_ERROR               = 633 | QL_FTP_ERRCODE_BASE,
QL_FTP_ERR_RECV_ERROR              = 634 | QL_FTP_ERRCODE_BASE,
} QL_FTP_CLIENT_ERR_E

```

● Member

Member	Description
<code>QL_FTP_ERR_OK</code>	Successful execution.
<code>QL_FTP_ERR_UNKNOWN</code>	Unknown error.
<code>QL_FTP_ERR_WOULDBLOCK</code>	The FTP(S) server is blocked.
<code>QL_FTP_ERR_BUSY</code>	The FTP(S) server is busy.
<code>QL_FTP_ERR_DNS</code>	Failed to parse domain name.
<code>QL_FTP_ERR_NETWORK</code>	Failed to establish socket or the network is deactivated.
<code>QL_FTP_ERR_CTRL_CLOSE</code>	The FTP session is closed by the FTP server.
<code>QL_FTP_ERR_DATA_CLOSE</code>	The data connection is closed by the FTP server.
<code>QL_FTP_ERR_BEARER_FAIL</code>	The socket is closed by peer.
<code>QL_FTP_ERR_TIMEOUT</code>	Timeout error.
<code>QL_FTP_ERR_INVALID_PARAM</code>	Invalid parameter.
<code>QL_FTP_ERR_FILE_NOT_FOUND</code>	Failed to open the file.
<code>QL_FTP_ERR_FILE_POINT_ERR</code>	Invalid file position.
<code>QL_FTP_ERR_FS_FILE_ERROR</code>	File error.
<code>QL_FTP_ERR_SERVICE_END</code>	The service is not available and control connection is closed.

<i>QL_FTP_ERR_DATA_FAILED</i>	Failed to open data connection.
<i>QL_FTP_ERR_CLOSE_CONN</i>	The connection is closed and transfer is aborted.
<i>QL_FTP_ERR_FILE_UNOPER</i>	Requested file action is not taken.
<i>QL_FTP_ERR_REQUEST_ABORT</i>	Requested action is aborted: local error in processing.
<i>QL_FTP_ERR_OVER_MEMORY</i>	Requested action is not taken: insufficient system storage.
<i>QL_FTP_ERR_COMMAND_ERROR</i>	Syntax error, command unrecognized.
<i>QL_FTP_ERR_PARAM_ERROR</i>	Syntax error in parameters or arguments.
<i>QL_FTP_ERR_COMMAND_FAILED</i>	The command is not implemented.
<i>QL_FTP_ERR_COMMAND_BAD_SEQUENCE</i>	Bad sequence of commands.
<i>QL_FTP_ERR_COMMAND_NOT_IMPLMENT</i>	The command parameter(s) is not implemented.
<i>QL_FTP_ERR_UNLOGIN</i>	Not logged in.
<i>QL_FTP_ERR_NO_ACCOUNT</i>	Need account for storing files.
<i>QL_FTP_ERR_REQUEST_FAILED</i>	Requested action is not taken.
<i>QL_FTP_ERR_REQUEST_STOP</i>	Requested action is aborted: page type unknown.
<i>QL_FTP_ERR_FILEREQ_STOP</i>	Requested file action is aborted.
<i>QL_FTP_ERR_FILENAME_ERROR</i>	Requested file name is invalid.
<i>QL_FTP_ERR_SSL_AUTH_FAIL</i>	SSL authentication failed.
<i>QL_FTP_ERR_INTERFACE_FAILED</i>	Source IP address for transmission cannot use.
<i>QL_FTP_ERR_SEND_ERROR</i>	Send data failed.
<i>QL_FTP_ERR_RECV_ERROR</i>	Receive data failed.

3.3.2. ql_ftp_client_new

This function creates an FTP(S) client handle and initializes FTP(S) client resources.

● Prototype

```
void *ql_ftp_client_new(void)
```

- **Parameter**

None

- **Return Value**

FTP(S) client handle Successful execution
NULL Failed execution

3.3.3. ql_ftp_client_release

This function releases FTP(S) client resources.

- **Prototype**

```
void ql_ftp_client_release(void *client)
```

- **Parameter**

client:

[In] FTP(S) client handle, i.e., the non-null value returned after *ql_ftp_client_new()* was executed successfully.

- **Return Value**

None

3.3.4. ql_ftp_client_setopt

This function configures FTP(S) client attributes.

- **Prototype**

```
int ql_ftp_client_setopt(void *client, QL_FTP_CLIENT_OPT_E tag, ...)
```

- **Parameter**

client:

[In] FTP(S) client handle.

tag:

[In] Attribute tags. See **Chapter 3.3.4.1** for details.

...:

[In] Attribute parameters. See **Chapter 3.3.4.1** for details.

● Return Value

See **Chapter 3.3.1.1** for error codes. When the return value is `QL_FTP_CLIENT_ERR_UNKNOWN`, please check whether *tag* and the attribute parameters after *tag* are the parameters specified in `QL_FTP_CLIENT_OPT_E` (**Chapter 3.3.4.1**).

3.3.4.1. QL_FTP_CLIENT_OPT_E

The enumeration of attribute tags or attribute parameters:

```
typedef enum {
    QL_FTP_CLIENT_SIM_CID,
    QL_FTP_CLIENT_SSL_CTX,
    QL_FTP_CLIENT_SSL_ENABLE,
    QL_FTP_CLIENT_OPT_PDP_CID,
    QL_FTP_CLIENT_OPT_SSL_CTX,
    QL_FTP_CLIENT_OPT_START_POS,
    QL_FTP_CLIENT_SSL_VERIFY_LEVEL,
    QL_FTP_CLIENT_SSL_CACERT_PATH,
    QL_FTP_CLIENT_SSL_OWNCERT_PATH,
    QL_FTP_CLIENT_SSL_CIPHERSUITE,
    QL_FTP_CLIENT_SSL_VERSION,
    QL_FTP_CLIENT_SSL_TRANSPORT
    QL_FTP_CLIENT_PORT_MODE,
    QL_FTP_CLIENT_IPV6_EXTEND,
    QL_FTP_CLIENT_ERR_FORMAT,
    QL_FTP_CLIENT_TIMEOUT,
}QL_FTP_CLIENT_OPT_E
```

● Member

Member	Description
<code>QL_FTP_CLIENT_SIM_CID</code>	The CID of (U)SIM card. It is obtained by binding the (U)SIM card (nSim) and PDP context ID (profile_idx) through <code>ql_bind_sim_and_profile()</code> . See document [3] for details.
<code>QL_FTP_CLIENT_SSL_CTX</code>	The SSL context used by the FTP(S) client.
<code>QL_FTP_CLIENT_SSL_ENABLE</code>	Whether the FTP(S) client uses SSL connection or not. <u>0</u> Connect to the FTP server <u>1</u> Implicitly encrypted connection to the FTPS server <u>2</u> Explicitly encrypted connection to the FTPS server <u>3</u> Connect to the SFTP server.

<code>QL_FTP_CLIENT_OPT_PDP_CID</code>	The PDP context ID used by the FTP(S) client, i.e., the PDP_CID used while executing a data call. See document [3] for details.
<code>QL_FTP_CLIENT_OPT_SSL_CTX</code>	SSL context.
<code>QL_FTP_CLIENT_OPT_START_POS</code>	The starting offset of the download.
<code>QL_FTP_CLIENT_SSL_VERIFY_LEVEL</code>	SSL verification method. See document [4] for details.
<code>QL_FTP_CLIENT_SSL_CACERT_PATH</code>	SSL certificate.
<code>QL_FTP_CLIENT_SSL_OWNCERT_PATH</code>	SSL local certificate.
<code>QL_FTP_CLIENT_SSL_CIPHERSUITE</code>	SSL cipher suite.
<code>QL_FTP_CLIENT_SSL_VERSION</code>	SSL version. See document [4] for details.
<code>QL_FTP_CLIENT_SSL_TRANSPORT</code>	SSL protocol type. See document [4] for details.
<code>QL_FTP_CLIENT_PORT_MODE</code>	Set active/passive mode of the FTP(S) client. <code>0</code> Active mode <code>1</code> Passive mode
<code>QL_FTP_CLIENT_IPV6_EXTEND</code>	Whether to enable IPV6 extension feature of the FTP(S) client. <code>0</code> Disable IPV6 extension feature <code>1</code> Enable IPV6 extension feature
<code>QL_FTP_CLIENT_ERR_FORMAT</code>	Error code format used by FTP(S) client. <code>0</code> Old error code format <code>1</code> New error code format
<code>QL_FTP_CLIENT_TIMEOUT</code>	Set FTP response timeout. Unit: seconds. Range: 20–180. Default value: 90.

3.3.5. ql_ftp_client_open

This function connects to the FTP(S) server.

● Prototype

```
int ql_ftp_client_open(void *client, char *hostname, char *username, char *password)
```

● Parameter

client:

[In] FTP(S) client handle.

hostname:

[In] FTP(S) server address. Maximum length: 200 bytes.

username:

[In] Username. Maximum length: 255 bytes.

password:

[In] Password. Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.6. ql_ftp_client_close

This function disconnects from the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_close(void *client)
```

- **Parameter**

client:

[In] FTP(S) client handle.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.7. ql_ftp_client_get_ex

This function downloads a file from the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_get_ex(void *client, char *remotefile, char *localfile, QL_FTP_CLIENT_WRITE_CB_EX  
write_cb, void* user_data)
```

- **Parameter**

client:

[In] FTP(S) client handle.

remoteFile:

[In] The name of the file to be obtained from the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

localFile:

[In] The name of the local file to be stored. It can be a relative path or an absolute path. Maximum length: 255 bytes.

If *write_cb* is not NULL, the parameter is invalid.

If *write_cb* and *localFile* are NULL, the local file name is the same as the file name on the FTP server.

If *write_cb* is NULL but *localFile* is not, the file is downloaded to the local path specified by user through the pointer *localFile*.

The default local directory is *UFS*:

write_cb:

[In] Callback function that receives the file data obtained from the FTP(S) server and returns the size of the received data. See **Chapter 3.3.7.1** for details.

If *write_cb* is not NULL, the file downloaded from the FTP(S) server will no longer be written to *localFile*, and users can determine what *user_data* is obtained from the FTP(S) server through the customized *write_cb*.

If *write_cb* is NULL, the file downloaded from the FTP(S) server will be written to *localFile*.

user_data:

[In] User data.

● Return Value

See **Chapter 3.3.1.1** for error codes.

3.3.7.1. QL_FTP_CLIENT_WRITE_CB_EX

This callback function receives the file data obtained from the FTP(S) server and returns the size of the received data.

● Prototype

```
typedef size_t (*QL_FTP_CLIENT_WRITE_CB_EX)(void *ptr, size_t size, size_t nmemb, void *stream)
```

● Parameter

ptr:

[In] Pointer to the data to be obtained. The obtained data can be written to the storage location specified by the user through this pointer.

size:

[In] The size of each element to be written. Unit: byte.

nmemb:

[In] The number of elements to be written. Final size of the data to be written: $size * nmemb$.

stream:

[In] The callback function parameter pointer passed by users. The pointer can be used to specify the location of the data stored in *ptr*.

● Return Value

Size of the written data	Successful execution
Values less than 0	Failed execution. FTP(S) ends abnormally.

3.3.8. ql_ftp_client_put_ex

This function uploads a local file or user data to the FTP(S) server.

● Prototype

```
int ql_ftp_client_put_ex(void *client, char *localfile, char *remotefile, QL_FTP_CLIENT_READ_CB_EX read_cb, void *user_data)
```

● Parameter

client:

[In] FTP(S) client handle.

localfile:

[In] The name of the local file to be uploaded. It can be a relative path or an absolute path. Maximum length: 255 bytes.

If *read_cb* is not NULL, the parameter is invalid.

If *read_cb* is NULL, *localfile* cannot be NULL. Otherwise, this function returns an error.

The default local directory is *UFS*:

remotefile:

[In] The name of the file to be stored on the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

If *remotefile* is NULL, the local file name is the same as the file name on the FTP(S) server.

If *read_cb* is not NULL, *remotefile* needs to be set. Otherwise, this function returns an error.

read_cb:

[In] The callback function used for sending data to the FTP(S) server. See **Chapter 3.3.8.1** for details.
If *read_cb* is not NULL, *localfile* is invalid and no local file is uploaded. Users can determine what *user_data* is to be uploaded through customized *read_cb*.

user_data:

[In] User data.

● Return Value

See **Chapter 3.3.1.1** for error codes.

3.3.8.1. QL_FTP_CLIENT_READ_CB_EX

The callback function sends data to the FTP(S) server.

● Prototype

```
typedef size_t (*QL_FTP_CLIENT_READ_CB_EX)(void *ptr, size_t size, size_t nmemb, void *stream)
```

● Parameter

ptr:

[Out] Pointer to the data to be read. The data passed by users can be read with this pointer to complete the record of *user_data*.

size:

[In] The size of each element to be read. Unit: byte.

nmemb:

[In] The number of elements to be read. Final size of the data to be read: *size*nmemb*.

stream:

[In] The callback function parameter pointer passed by users. The pointer can be used to pass the data to *ptr*.

● Return Value

Size of the data to be read	Successful execution
Values less than 0	Failed execution. FTP(S) ends abnormally.

3.3.9. ql_ftp_client_delete

This function deletes an FTP(S) server file.

- **Prototype**

```
int ql_ftp_client_delete(void *client, char *remotefile)
```

- **Parameter**

client:

[In] FTP(S) client handle.

remotefile:

[In] The name of the FTP(S) server file to be deleted. It can be a relative path or an absolute path.
Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.10. ql_ftp_client_cwd

This function alters the current directory used by the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_cwd(void *client, char *path)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[In] FTP(S) server directory. It can be a relative path or an absolute path. Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.11. ql_ftp_client_lcwd

This function alters the local directory currently used.

- **Prototype**

```
int ql_ftp_client_lcwd(void *client, char *path)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[In] Local directory. It can be a relative path or an absolute path. Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.12. ql_ftp_client_pwd

This function gets the current directory used by the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_pwd(void *client, char *path, int path_len)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[Out] FTP(S) server directory. It is an absolute path. Maximum length: 255 bytes.

path_len:

[In] The maximum storage length of *path*.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.13. ql_ftp_client_lpwd

This function gets the local directory currently used.

- **Prototype**

```
int ql_ftp_client_lpwd(void *client, char *path, int path_len)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[Out] Local directory. It is an absolute path. Maximum length: 255 bytes.

path_len:

[In] The maximum storage length of *path*.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.14. ql_ftp_client_mkdir

This function creates a directory on the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_mkdir(void *client, char *path)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[In] The FTP(S) server directory to be created. It can be a relative path or an absolute path. Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.15. ql_ftp_client_rmdir

This function deletes the directory on the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_rmdir(void *client, char *path)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path:

[In] The FTP(S) server directory to be deleted. It can be a relative path or an absolute path. Maximum length: 255 bytes.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.16. ql_ftp_client_list

This function gets the list of the specified file or directory on the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_list(void *client, char *path_or_file, char *match, QL_FTP_CLIENT_FLIE_INFO_T *file_info_list, int max_file_num)
```

- **Parameter**

client:

[In] FTP(S) client handle.

path_or_file:

[In] The directory or file on the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

match:

[In] Match characters listing files that contains the character *match*.

file_info_list:

[Out] The list of files and directories obtained. See **Chapter 3.3.16.1** for details.

max_file_num:

[In] The maximum number of files that can be stored in *file_info_list*.

● Return Value

Greater than 0 The number of the list actually obtained

-1 Failed execution

3.3.16.1. QL_FTP_CLIENT_FILE_INFO_T

The structure of the files and directories:

```
typedef struct{
    char filename[MAX_FILE_NAME_SIZE];
    int type;
    int permissions;
    int links;
    char owner[MAX_OWNER_SIZE];
    char group[MAX_OWNER_SIZE];
    struct tm timestamp;
    unsigned long size;
}QL_FTP_CLIENT_FILE_INFO_T
```

● Parameter

Type	Parameter	Description
char	<i>filename</i>	File or directory.
int	<i>type</i>	Type. 0 Unknown 1 Directory 2 File
int	<i>permissions</i>	Permissions.
int	<i>links</i>	Links.
char	<i>owner</i>	Owner.
char	<i>group</i>	Owner group.
<i>struct tm</i>	<i>timestamp</i>	Creation time.
unsigned long	<i>size</i>	File size. Unit: byte.

3.3.17. ql_ftp_client_size

This function gets the size of the specified file under the specified path on the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_size(void *client, const char *filename, double *nSize)
```

- **Parameter**

client:

[In] FTP(S) client handle.

filename:

[In] The file or directory on the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

nSize:

[Out] The size of the specified file obtained from the FTP(S) server.

- **Return Value**

See **Chapter 3.3.1.1** for error codes.

3.3.18. ql_ftp_client_rename

This function renames the specified file or directory on the FTP(S) server.

- **Prototype**

```
int ql_ftp_client_rename(void *client, const char *oldname, const char *newname)
```

- **Parameter**

client:

[In] FTP(S) client handle.

oldname:

[In] The current directory name or file name on the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

newname:

[In] The new name of the file or directory currently specified by *oldname* on the FTP(S) server. It can be a relative path or an absolute path. Maximum length: 255 bytes.

- Return Value

See *Chapter 3.3.1.1* for error codes.

4 Example

For detailed information about FTP(S) example, see *ftp_demo.c* and *ftp_demo2.c* example files in `\components\ql-application\ftp` directory in the application in CSDK and *sftp_demo.c* example file in `\components\ql-application\sftp` directory.

5 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_RTOS_Development_Guide
[3] Quectel_EC200U&EG91xU_Series_QuecOpen_Data_Call_API_Reference_Manual
[4] Quectel_EC200U&EG91xU_Series_QuecOpen_SSL_API_Reference_Manual

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CID	Customer Identity
DNS	Domain Name Server
FTP	File Transfer Protocol
FTPS	FTP over SSL
IoT	Internet of Things
RTOS	Real-Time Operating System
SDK	Software Development Kit
(U)SIM	(Universal) Subscriber Identification Module
SSL	Secure Sockets Layer
TLS	Transport Layer Security